

Coordinated multi-agent software development — Workflow, roles and dashboard

Confluence page 3227216050 · version 5 · updated 2026-10-01 10:59:51 UTC
Captured 2026-10-01T13:59:15.885Z · [Original Confluence page](#)

Historical source export. Development states are retained as written in version 5. This PDF is locally typeset from Confluence export HTML with original figures; it is not Confluence's native PDF export. Updated workflow lessons are in the repository guide.

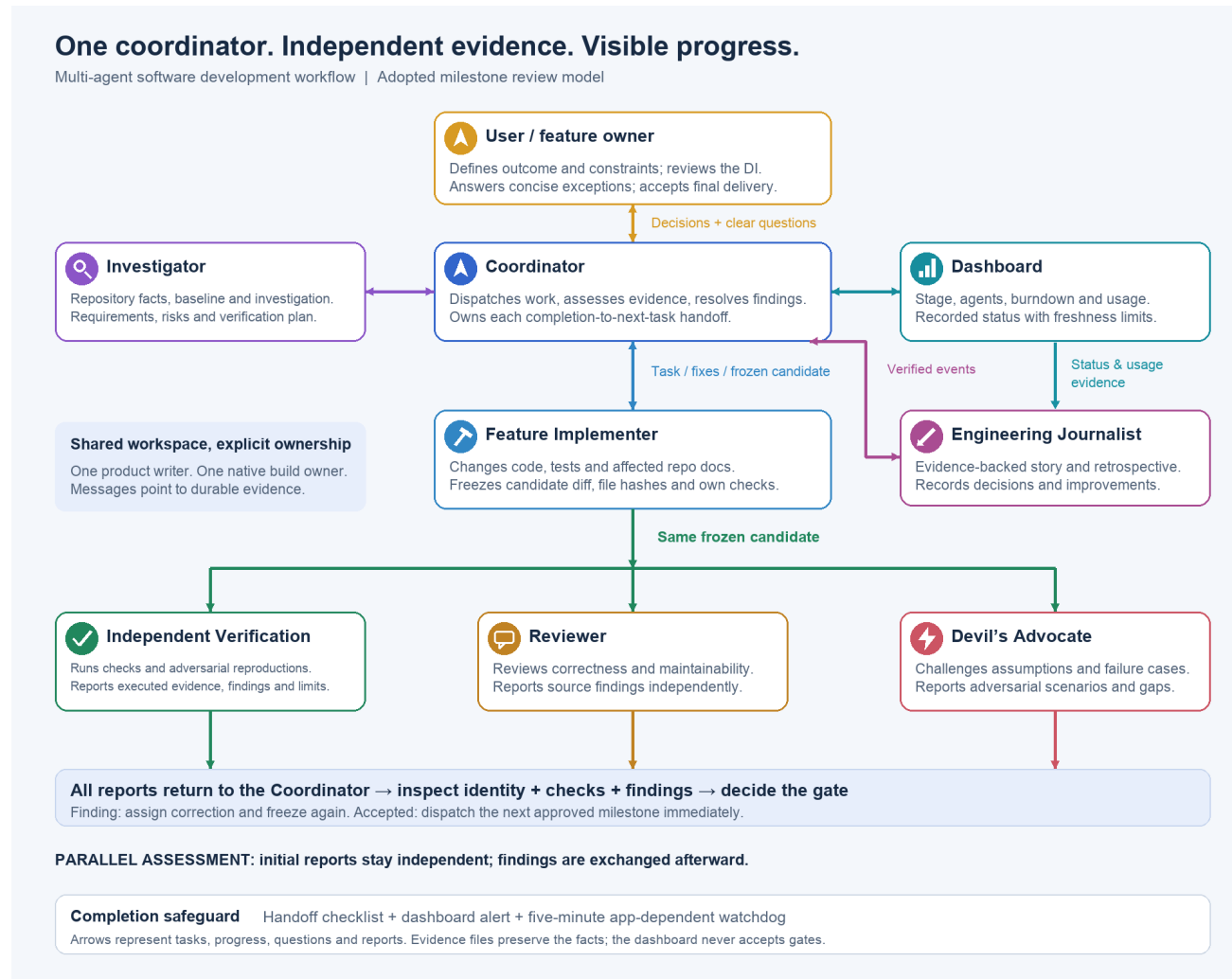
Coordinated multi-agent software development

One feature owner, one accountable coordinator, seven specialist roles. The workflow turns a feature brief into an approved design, a series of identified and independently assessed milestones, accurate repository documentation and a reviewable delivery. The dashboard makes the current stage, next handoff and required user decision visible.

This is an evolving working practice around Codex agents, a controlled repository and a locally hosted dashboard. The Herdr *Go to popup configurability* project provides the examples below; it is still in development. This page explains the adopted workflow and separates actual observations from illustrative figures.

- [1. The team and its communication paths](#)
- [2. When the user is needed](#)
- [3. From approved design to a verified milestone](#)
- [4. What agents exchange](#)
- [5. A dashboard designed for a quick glance](#)
- [6. Preventing a completed task from becoming a stalled feature](#)
- [7. A concrete example: green checks, but the gate stays open](#)
- [8. Delivery, documentation and learning](#)
- [9. Demonstration guide and references](#)

1. The team and its communication paths



How to read the figure: the Coordinator sends bounded assignments and receives progress, questions and completion reports. The Implementer freezes a candidate; the Coordinator dispatches that same state to three independent assessors. Reports return to the Coordinator for a gate decision. Dashboard and Journalist receive evidence-backed events. The Dashboard → Journalist arrow shows how recorded status and usage evidence inform the engineering narrative through durable records; it does not imply an automated feed. Arrows represent communication and handoffs, not permission for concurrent source changes.

Role	Responsibility and output	Ownership / boundary
 Coordinator	Keeps the whole feature moving. Tracks scope, prerequisites, candidate identity, gate evidence, findings, resources and the next owner. Asks concise user questions when a decision is needed.	Accountable for every completion-to-next-task handoff and final delivery.
 Investigator	Reads repository behavior and contracts, establishes a baseline and prepares the detailed investigation (DI), design choices, risks, requirement traceability and ordered tasklist.	Investigation first; no implied implementation or release authority.
 Feature Implementer	Writes feature code, tests, configuration examples and affected repository documentation. Runs own checks, fixes accepted findings and freezes candidate evidence.	Sole product source and feature-doc writer in this project.
 Independent Verification	Runs native requirement-based checks, adversarial inputs and interaction reproductions against an identified frozen candidate. Reports results, logs, findings and coverage limits.	Owens native builds/caches during assessment; own implementation tests are not independent signoff.
 Reviewer	Independently assesses correctness, design, integration and maintainability. Reports actionable source findings with concrete scenarios.	Read-only during the frozen gate; does not compete for the native build resource.
 Devil's Advocate	Challenges assumptions, missing cases and failure boundaries. Turns concerns into concrete counterexamples or the smallest resolving checks.	Independent initial criticism, followed by evidence-based discussion.
 Dashboard	Maintains stage, assignments, next handoffs, burndown, findings, evidence links and scoped usage measurements.	Sole dashboard owner; displays recorded state and freshness, never accepts a product gate.
 Engineering Journalist	Maintains the factual engineering story, decision history and attribution. Supports the demonstration narrative and actual team retrospective.	Narrative ownership; product documentation remains the Implementer's responsibility.

2. When the user is needed

The user supplies direction and approves consequential choices. Routine implementation, testing, review exchanges and fixes within the approved scope move forward without repeated user approval. The user can consult or redirect the Coordinator at any time.

User touchpoint	What the user provides or reviews	When interaction is needed
 Feature definition	Desired behavior, repository/base, examples, constraints, acceptance concerns and excluded work.	Required at kickoff; a concise clarification only when missing information changes the outcome.
 DI output review	Design choices, compatibility behavior, requirements, risks, task dependencies, checks and delivery boundary.	Required before implementation in this workflow. Approve or revise the concrete package.
 Exception or scope decision	A specific unresolved choice, unavailable required check, material scope change or authority outside the current instruction.	Conditional. Ask one clear question with recommendation and practical consequence; do not ask the user to manage internal handoffs.
 Milestone observation	Dashboard, identified candidate, review findings and evidence.	Optional. Technical gate acceptance belongs to the Coordinator within the approved plan.
 Final acceptance	Reviewable feature, repo documentation, executed evidence, remaining limits and retrospective.	User reviews delivery. Commit, push, merge, install and deployment authority remain separate unless explicitly granted.

Example of a useful question

Decision needed: May we deliver this as a locally validated macOS candidate with Windows checks explicitly unrun? **Recommendation:** yes for local review; Windows release readiness remains unclaimed.

3. From approved design to a verified milestone

Feature brief → **investigation** → **DI approval** → **implementation** → **frozen candidate** → **three parallel assessments** → **finding disposition** → **next milestone** → **final integrated assessment** → **delivery and retrospective**.

Step	Concrete handoff / evidence
1 · Dispatch	The Coordinator specifies goal, acceptance criteria, scope, exact source state, mutation authority, resources, expected evidence and stop condition.
2 · Implement	The Implementer changes source, tests and affected docs; runs appropriate own checks and reports what is ready.
3 · Freeze and identify	Source edits stop. Record exact base, complete scoped patch, new-file inventory and relevant file hashes. Without commit authority, these records identify the local candidate.

Step	Concrete handoff / evidence
4 · Assess in parallel	Verification executes checks; Reviewer inspects source; Devil's Advocate challenges failure boundaries. Initial reports are frozen before other assessors' conclusions are shared.
5 · Discuss findings	After independent reports, participants exchange concrete triggers and evidence. Record severity, requirement, impact, fix owner and acceptance disposition.
6 · Correct or accept	A confirmed defect returns to the Implementer. Freeze the corrected state and recheck affected behavior. Accept only when all required reports and findings are accounted for.
7 · Continue immediately	Acceptance must dispatch the next approved milestone. A completed assignment is not the same as a completed feature.

Parallel reasoning, controlled mutations: read-only review can overlap with verification. Only one actor owns shared native build outputs, caches and services at a time. Frozen product source does not change underneath an assessor. The assessment trio now applies to frozen milestones from C2 onward, as requested by the user, and to the final integrated candidate.

Pipeline view: work, waits and bottlenecks

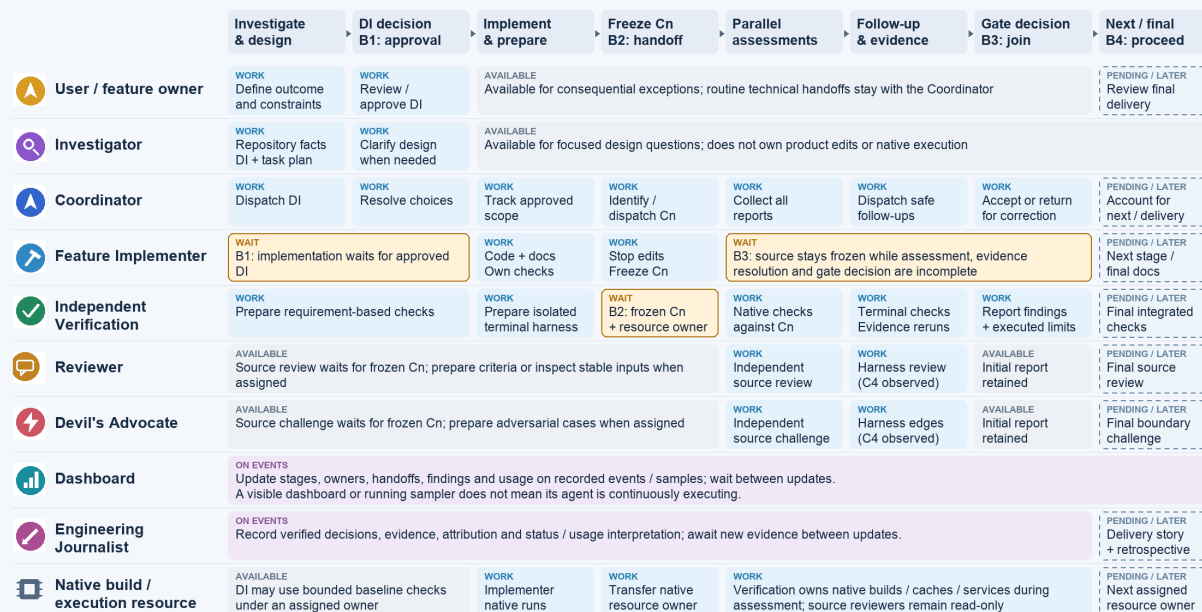
Read this like a processing pipeline: each lane is a role or a shared execution resource, and work advances only when its inputs are ready. The chart shows which activities can overlap and which waits protect approval, candidate identity, independent assessment or shared resources. It helps identify the dependency to unblock, rather than treating every idle agent as a shortage of workers.

Agent pipeline: work, waits and useful overlap

One candidate cycle, left to right. Find the prerequisite that keeps a lane waiting.

Illustrative schedule, not an execution log: equal-width phases do not represent elapsed time or hardware utilization.

☐ Active task
 ☐ Blocked by prerequisite
 ☐ Available for safe work
 ☐ Evidence-driven update
 ☐ Later / not completed



Correction loop Confirmed defect → Implementer corrects → freeze a new candidate → reassess → decide the gate

Safe overlap requires stable inputs and explicit ownership. A waiting lane is not permission to edit frozen source.

Observed C4 example • development-chat reports, 1 October 2026

Reviewer and Devil's Advocate finished source reviews while Verification continued terminal checks. The Coordinator then assigned both read-only harness reviews in parallel; those reviews found COPY, return-to-pane and cleanup evidence gaps. T4 remains open at this observation.

Bottlenecks to inspect before adding more workers

1 Approval / handoff wait

Is a concrete decision or identified candidate ready, but the next owner has not been dispatched? Measure decision and dispatch delay.

2 Shared native execution

Only one owner may mutate shared build outputs and services. Keep read-only review parallel; isolated runners are a future option to qualify.

3 Long evidence tail / rework

Source reviews can finish before runtime checks. Assign safe evidence review; reduce harness ambiguity and rerun time without weakening gates.

Optimization ideas are proposals. Agent-token use and local CPU / memory are different resources; no utilization percentage is claimed here.

Figure: an illustrative scheduling map for one candidate cycle. Columns show ordered phases, not measured elapsed time; bar widths do not estimate duration or utilization. Blue marks work, amber marks a prerequisite wait, gray marks availability, purple marks updates driven by recorded events, and dashed blocks mark later work. The existing colorful role icons are reused. [Open the full-size pipeline diagram.](#)

Dependency / bottleneck	Who waits, and for what?	Useful overlap or improvement	What to measure in a future trace
B1 · DI approval	Implementation waits for the user's decision on the concrete DI package.	Prepare checks and stable baseline inputs; clarify unresolved decisions. Product changes still require approval.	DI ready → decision → implementation dispatch. Separate decision time from missed handoff time.
B2 · Candidate and resource handoff	Assessors need identified frozen source; Verification also needs exclusive ownership of shared native outputs, caches and services.	Prepare isolated harnesses and review criteria while implementation runs. At freeze, dispatch all three initial assessments together.	Freeze → each assessor dispatched / started; resource ready → execution start; queue time by owner.
B3 · Assessment join and evidence tail	Implementer source remains frozen until every required report and finding disposition is complete. Runtime execution may outlast source review.	Use available reviewers for assigned read-only harness / evidence checks after preserving initial independent reports. Correct ambiguity and recheck evidence.	Each assessment duration; time from first report to last required evidence; reruns and their causes.
B4 · Gate-to-next-task handoff	The next approved stage waits for gate acceptance and an explicit assignment. A confirmed defect loops through correction, a new freeze and reassessment.	Dispatch the next owner promptly when the gate is accepted. Preserve candidate identity and the one-writer rule.	Required reports complete → findings resolved → gate accepted → next owner started.
Shared execution capacity	Native mutation and performance work depend on qualified resources, not simply on the number of agents.	Proposed for T5: partition native tests/builds, documentation checks and isolated runtime checks where independent; keep one shared-cache owner, separate runtime sessions/sockets and uncontended performance measurements. This is a Coordinator proposal, not evidence of completed parallel qualification.	Busy time, resource queue time and execution overlap by resource; compare end-to-end completion and rework.

Reported C4 example, observed 1 October 2026 at 12:40 CEST: Reviewer and Devil's Advocate completed their initial source reviews while Verification continued terminal checks. They then reviewed the terminal harness in parallel and identified COPY, return-to-pane and cleanup evidence gaps. The Coordinator's latest update says one focused C4 interaction check remains; T4 is still open at this cutoff. These reports demonstrate useful overlap, not a measured hardware-utilization gain or a confirmed product defect.

Future bottleneck instrumentation: retain role, task, candidate, resource owner, ready/start/end times, blocked-by dependency, wait reason and rework cause. Distinguish prerequisite waits, resource queues, available capacity and unassigned ready work. Measure actual intervals before ranking delays; optimize the longest dependency path to delivery. Local CPU/RSS and cloud-model token usage cover different resources. This trace and any utilization percentages are proposed; the diagram is not live telemetry.

Execution-time view: the milestone bottleneck

The comprehensive pipeline above explains ownership and dependencies. This companion figure focuses on **Implementer, Verification, Reviewer and Devil's Advocate**, with time-scaled activity bars and interactions. A long upstream implementation interval starves the assessors; a long downstream verification interval holds the next implementation milestone at its gate.

Execution time: find the lane holding development throughput

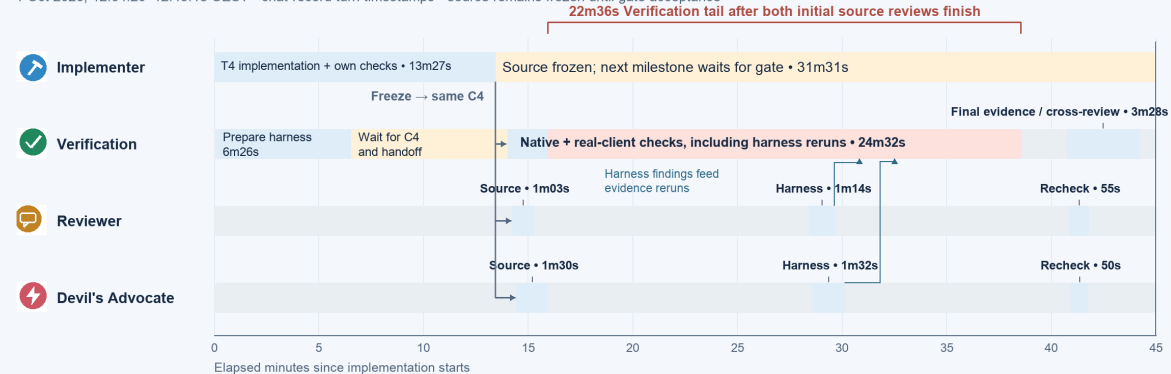
Four agents on a common elapsed-time axis. The longest required path holds the next milestone.

A is measured C4 turn wall-clock time; B is a hypothetical implementation bottleneck. Neither shows CPU or cloud-compute utilization.

Active task Critical-path tail Dependency / gate wait Outside captured active turns

A. Observed C4: Verification outlasts both initial source reviews

1 Oct 2026, 12:04:20–12:49:18 CEST • chat-record turn timestamps • source remains frozen until gate acceptance

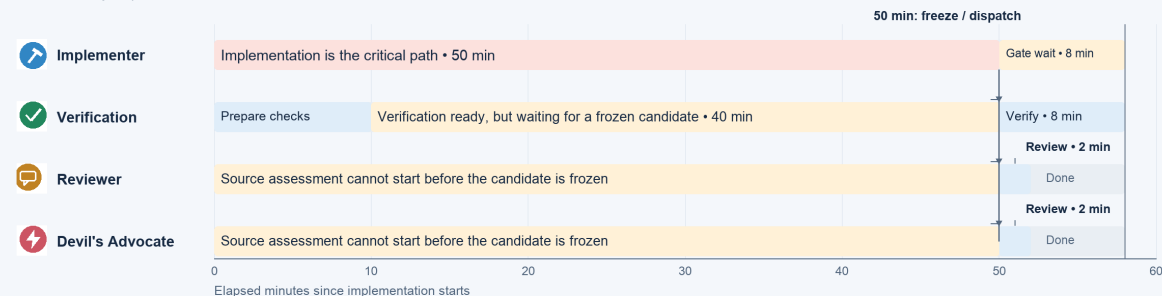


Next stage starts at 44m58s. The 31m31s Implementer wait includes checks, evidence follow-up, gate handling and dispatch.

Durations are rounded elapsed agent turns, including tool execution / waits and harness correction. They are not model-only runtimes.

B. Hypothetical: slow implementation starves all three downstream assessors

Illustration only: implementation 50 min; Verification 8 min; each source review 2 min. No defect or rework in this scenario.



Sensitivity in this hypothetical case

Implementation 50 → 25 min: gate 58 → 33 min. Verification 8 → 4 min: gate 58 → 54 min.

Optimize the longest required path. More workers help only when work is independent and integration overhead is controlled.

Proposals: split independent verification tasks with isolated runtimes and one shared-cache owner; or split implementation into bounded components with clear interfaces, isolated edits and one integration owner. Preserve independent signoff and keep performance measurements uncontended.

Keep the comprehensive pipeline figure for ownership rules; use this time-scaled view to diagnose milestone latency.

A • Measured C4 example: the axis runs from the Implementer's T4 assignment start at **12:04:20** to its T5 assignment start at **12:49:18 CEST on 1 October 2026**. Bar boundaries use chat turn start/end timestamps; duration labels use reported elapsed turn durations rounded to seconds. They include reasoning, tools, waiting and harness correction, rather than model-only or CPU runtimes. Gray means outside the captured active turns. Interaction arrows indicate evidence/dependency flow; their intermediate routing does not timestamp individual messages. [Open the full-size execution-time diagram](#).

Agent	C4 initial / main elapsed turn	Additional captured work
 Implementer	13m27s — T4 changes, own checks and candidate freeze.	Source frozen from 12:17:47 until the next assignment at 12:49:18: 31m31s of gate and handoff wait.
 Verification	24m32s — native and real-client checks, qualification and retained harness reruns.	6m26s harness preparation overlapped implementation; final evidence / cross-review took 3m28s .
 Reviewer	1m03s — initial independent source review.	1m14s harness supplement and 55s final cross-review.
 Devil's Advocate	1m30s — initial independent source challenge.	1m32s harness supplement and 50s final cross-review.

Where C4 was held: both initial source reviews were complete by **12:20:16**; Verification's main report finished at **12:42:52**, leaving a **22m36s verification tail**. Its final evidence report followed at 12:48:32. Verification and evidence closure therefore determined the assessment finish in this milestone. The Implementer's full 31m31s wait also includes report exchange, gate handling and dispatch; it must not all be attributed to executing tests. Read-only harness reviews overlapped Verification and helped strengthen the evidence. The completed C4 gate enabled T5; this is a historical measurement, not a current blocking-status claim.

B · Hypothetical implementation bottleneck: with 50 minutes of implementation, 8 minutes of Verification and two 2-minute source reviews starting together at freeze, the earliest gate is **58 minutes**, assuming no rework or handoff delay. Cutting implementation to 25 minutes moves that gate to **33 minutes**; cutting Verification to 4 minutes moves it only to **54 minutes**. This panel illustrates where optimization has the most effect; these are not observed implementation timings for Herdr.

- **When Verification dominates:** partition independent native checks, documentation checks and isolated runtime checks; qualify harnesses early and reduce ambiguous assertions and reruns. The development Coordinator has authorized a T5 verification split, but a speedup has not yet been measured. Keep one shared-cache owner, isolated runtime sessions and uncontended performance runs.
- **When implementation dominates:** a future option is to split independent components behind clear interfaces, use isolated editing areas and appoint one integration owner. Measure integration and rework costs before assuming that more Implementers improve delivery time.

Improvement criterion: shorten the longest required path to an accepted milestone while preserving independent checks. Compare end-to-end time, resource queues, gate waits, integration overhead and reruns. One C4 cycle establishes milestone latency, not sustained features-per-hour throughput. Timing provenance is read-only inspection of the four specialist chats in the development-chat tree; no development evidence records were modified.

4. What agents exchange

Exchange	Purpose
Coordinator ↔ specialist	Task dispatch, progress, a precise question or blocker, completion report and next action.

Exchange	Purpose
Frozen candidate → assessors	Same source identity and acceptance scope, with durable evidence references.
Assessor → Coordinator	Independent findings, actual executed checks where applicable, limits and gate recommendation.
Cross-role exchange after initial reports	Reproduction details, factual challenge, smallest correction and independent confirmation. Direct messages are bounded by the task and do not replace the evidence record.
Coordinator → Dashboard / Journalist	Verified workflow events, accepted gates, current owner, finding dispositions and real user decisions.
Dashboard → Journalist	Recorded status and usage evidence support the engineering story and retrospective, with timestamps, measurement assumptions and coverage limits retained. This is an evidence flow through records, not a claim of automated delivery.
All roles → retrospective	First-hand reflections, factual corrections, ranked improvement ideas and accountable follow-up actions.

Durable records distinguish **reported claims**, **source analysis** and **executed evidence**. A hash shows what was checked, not that a command ran. Test source is not a passing run; zero selected tests is not a pass. Findings retain attribution for discovery, reproduction, correction and independent recheck.

5. A dashboard designed for a quick glance

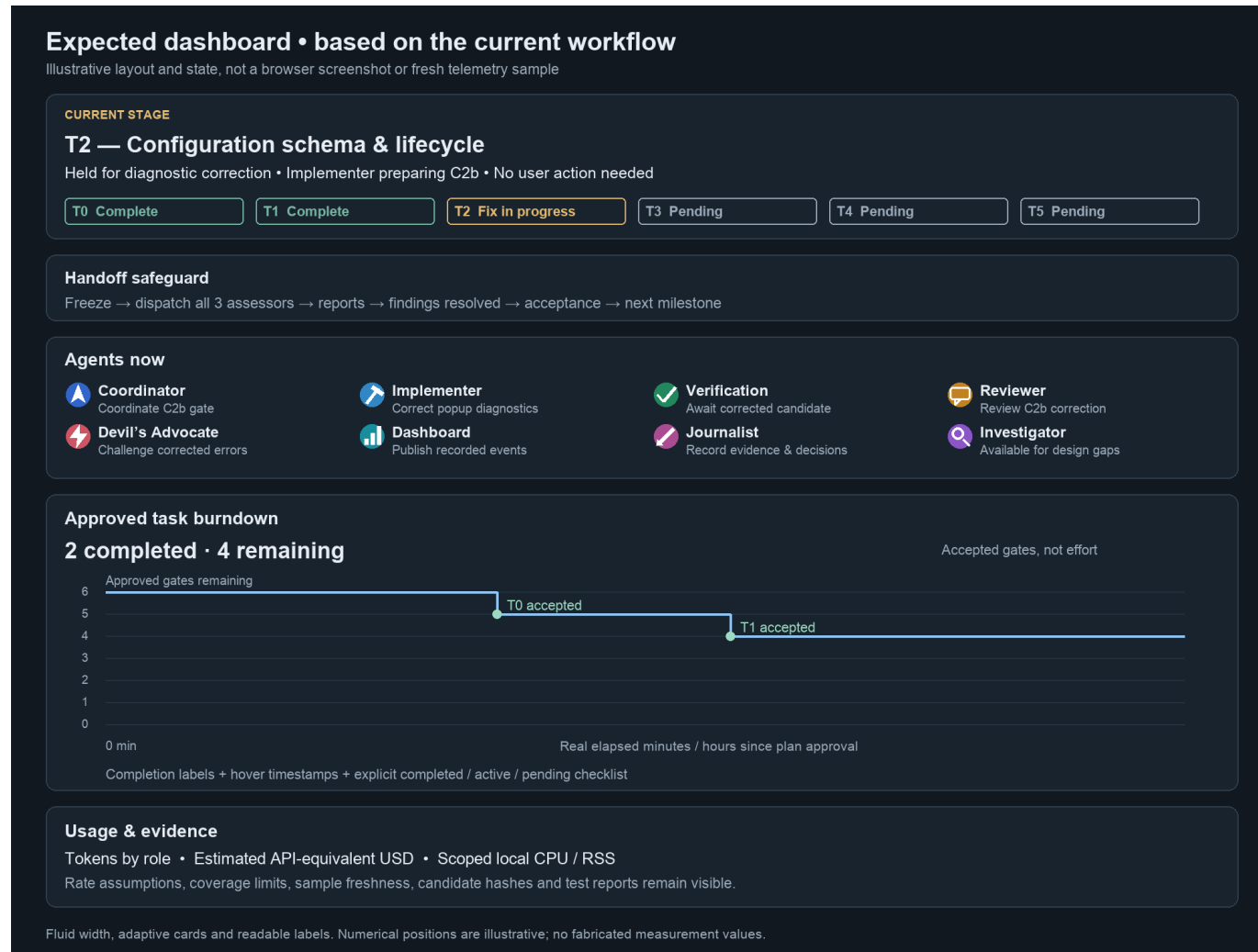
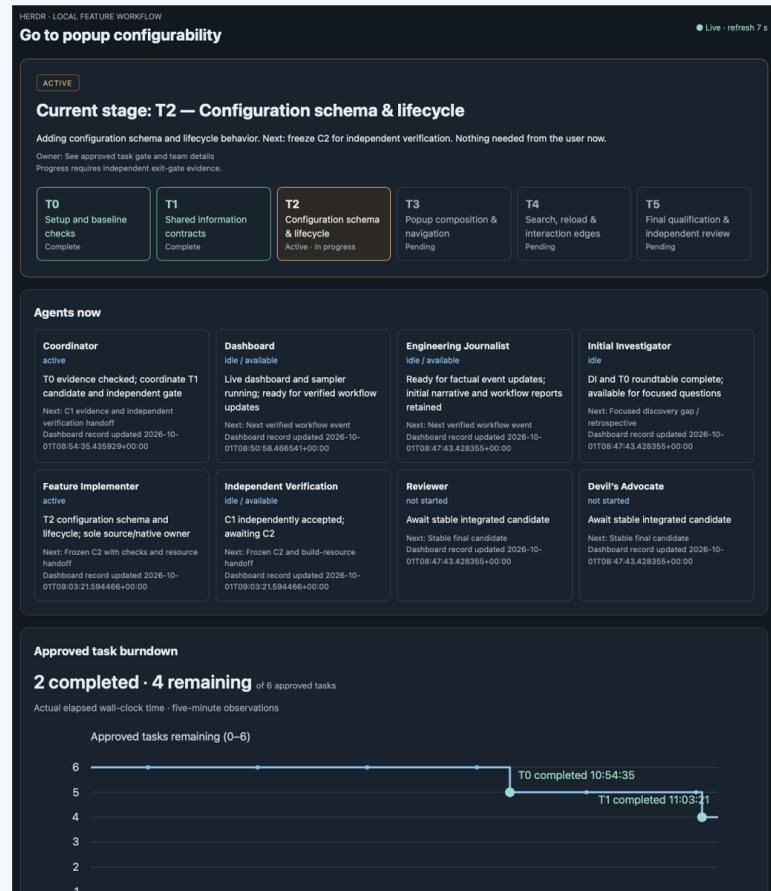


Figure: expected dashboard layout. This is an illustration based on the current dashboard sections and the C2 correction scenario, not a browser screenshot or a telemetry sample. Its numerical positions are illustrative. It shows the design intent after the fluid-width, usage and handoff improvements.

Visible section	What it should answer
Current stage and next action	Where are we? What is happening next? Is anything needed from the user?
Agents now	Which role owns the work, what is its assignment, what is it waiting for and when was its record updated?
Task burndown	Y-axis: approved gates remaining, an integer count. X-axis: actual elapsed minutes/hours since plan approval. A drop is an accepted gate; a flat line may include ongoing work. Completed, active and pending tasks remain listed.
Handoff safeguard	Has the candidate been frozen, dispatched to all required assessors, assessed, accepted or returned for correction, and handed to the next owner?
Review and evidence	Which candidate is under assessment, what findings remain and which checks actually ran?
Usage and freshness	Tokens by role, clearly labelled API-equivalent USD estimates, scoped local CPU/RSS, collection timestamps, rate assumptions and coverage gaps.

Dashboard example from the live Herdr feature workflow

Actual user-supplied capture, 1 October 2026 • cropped to dashboard content



Historical snapshot: captured before fluid-width layout, parallel milestone review, usage and handoff alert updates.

1 Current stage

Shows T2, the next handoff and whether user input is needed.

2 Approved milestones

T0 and T1 completed; T2 active. T3–T5 remain pending.

3 Agents now

Role, current assignment, next handoff and record timestamp.

4 Burndown

Y: approved gates remaining (0–6).
X: real minutes / hours since approval.

Reading the line

A drop means an accepted gate.
A flat line does not mean zero work.

Figure: actual dashboard example, 1 October 2026. Cropped from the user's supplied capture to show the project content. It predates the fluid-width layout, parallel milestone reviews, usage panel and handoff alert. It is a historical example, not the current feature status. A fresh browser capture was unavailable through the connector during preparation.

Usage interpretation: session-tree counters include repeated context and cached input; they are not unique text. Dollar values compare equivalent API token usage under published rate assumptions and are not the user's bill. Local process CPU/RSS covers registered task processes; it does not measure cloud model compute. Missing coverage is shown as unavailable, not zero. The locally hosted dashboard is accessible only on its originating workstation.

6. Preventing a completed task from becoming a stalled feature

This case exposed missed completion handoffs. The Coordinator acknowledged the failure and resumed the next assignments. The adopted safeguards are:

- **A candidate completion checklist:** freeze identity → dispatch each assessor → collect each report → dispose findings → accept or correct → dispatch the next owner.
- **A visible handoff alert:** show a required next action that lacks an active recorded owner, including record age. Stale dashboard records must not be presented as fresh agent status.
- **A five-minute coordination watchdog:** inspect actual agent completion states and the evidence record, then resume the next authorized handoff. It depends on the app and scheduler being available; it is recovery support, not a guaranteed real-time completion handler.
- **A final delivery checklist:** the feature, verification, reviews, affected repository docs and requested retrospective must all be accounted for.

The app's subagent panel shows active worker assignments. **"Active 0" means no subagent is currently executing; it does not prove feature completion.** The Coordinator may be assessing reports at that moment. If required work remains without a dispatched next action, that is a coordination defect. These new safeguards still need effectiveness checks in the retrospective; their existence does not establish reliable unattended operation.

7. A concrete example: green checks, but the gate stays open

In the Herdr C2 milestone, native schema/lifecycle checks and 36 independent configuration-check fixtures passed their stated expectations. Verification nevertheless reproduced an acceptance gap: malformed popup rows were rejected, but the startup error did not identify the `ui.goto` setting. The approved design required accurate popup diagnostic context.

Reviewer and Devil's Advocate independently completed their initial reviews, then both concurred with the reproduced finding. The Coordinator held T2, assigned a narrow correction, and dispatched the frozen C2b correction for all three follow-up assessments. C2b subsequently passed the independent native checks and 40 CLI fixtures; both source assessors were clear, and the Coordinator accepted T2 and dispatched T3. This shows why a gate depends on requirements and resolved evidence, not a green check count.

The project remains in development. This example does not claim later popup rendering, real-client interaction, complete platform validation or final release readiness. The evidence record and live dashboard carry current status; this page is a reusable workflow demonstration.

8. Delivery, documentation and learning

A delivery identifies the final candidate, approved behavior, affected files, accurate repository documentation, actual executed checks, resolved findings and material limits. The Implementer owns feature documentation in the repo; Verification checks its accuracy. The Journalist's engineering story remains a separate record of how the team reached the result.

After final verification and review, the actual participating agents hold a bounded retrospective: independent reflections, cross-role factual clarification, prioritized improvements and draft attribution checks. Follow-up actions name a responsible role, trigger, measurable outcome and effectiveness check. Participation gaps and disagreement stay visible. Publishing or deploying follows the user's destination-specific authority.

9. Demonstration guide and references

1. Start at the communication figure: identify the feature owner, accountable Coordinator and seven supporting roles.
2. Show the user decision table: feature definition and DI approval are the planned checkpoints; later questions are concise and conditional.
3. Walk through one frozen milestone: parallel independent reports, a concrete finding, correction and gate acceptance.
4. Open the dashboard example: current stage, next owner, readable burndown, evidence, usage and freshness.
5. Finish with delivery and retrospective: explain how repo documentation and workflow learning are owned.

[Herdr DI](#) — [Go to popup configurability](#) · [Herdr project collection](#) · [Herdr feature retrospectives](#)

The published DI is the reviewed design reference; historical execution wording inside it is not a live status feed. The local dashboard and current evidence record are the operational status sources. This demonstration imports no hardware-specific signoff requirements.

Prepared 1 October 2026, 11:46 CEST from the approved feature plan, user instructions, frozen milestone reports and actual dashboard artifacts. Figure assets are attached to this page for portable viewing; no local logs, session transcripts or repository archive are uploaded.